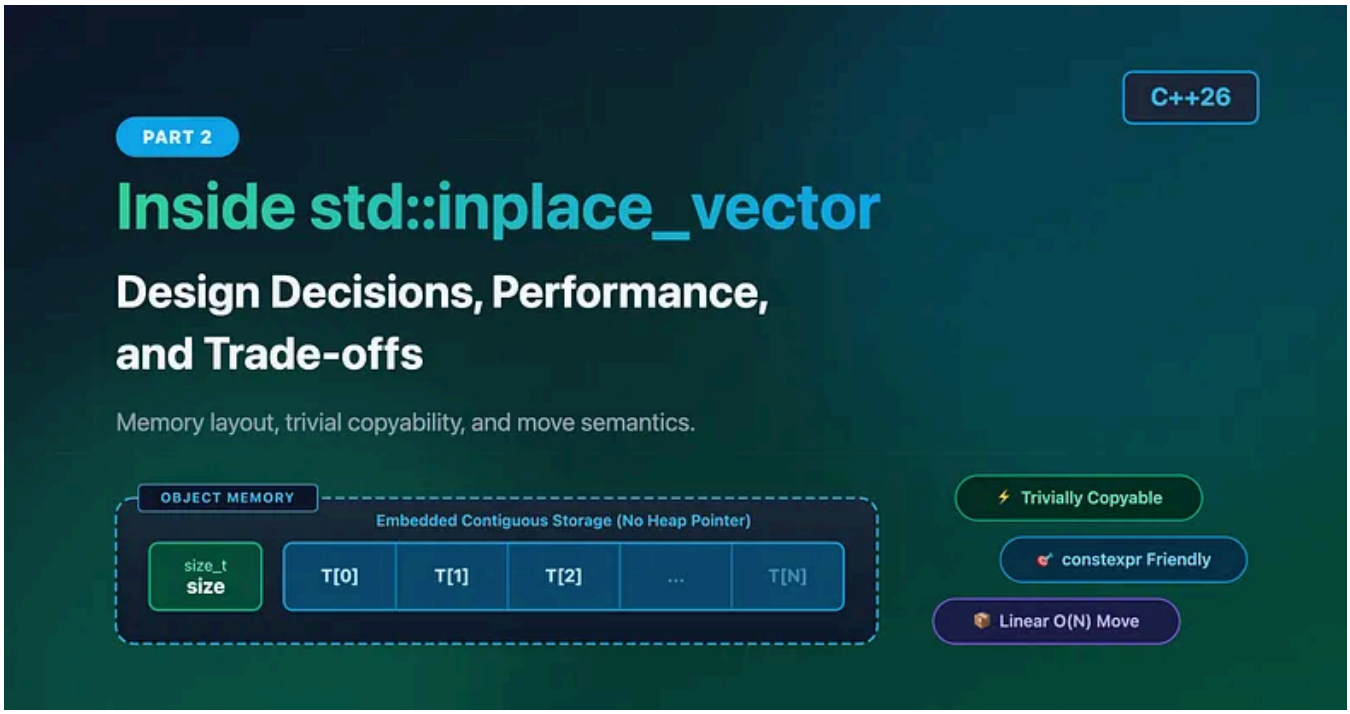
Part 2: Inside C++26 `std::inplace_vector` — Design Decisions, Performance, and Trade-offs

How C++26's fixed-capacity vector works under the hood — and why its design matters?

6 min read · 6 hours ago

Sagar Following

Listen Share More



In Part 1, we explored what `std::inplace_vector` is, how it differs from `std::vector` and `std::array`, and the API that C++26 brings to the standard library.

Up to this point, `std::inplace_vector` has looked like a `std::vector` with a fixed capacity. That's true from a user's perspective.

Internally, though, it's a completely different container.

The implementation choices made by the committee affect everything from move semantics to iterator invalidation, object lifetime, and even whether the type can be trivially copyable.

These aren't just implementation details — they influence when `std::inplace_vector` is the right tool and when it isn't.

. . .

The Storage Lives Inside the Object

The most important thing to remember about `std::inplace_vector` is this:

> *The elements are stored inside the container itself.*

That single decision changes almost everything. Consider these two containers.

```
std::vector<int> a;
std::inplace_vector<int, 8> b;
```

The `std::vector` object is tiny. It typically stores:

- a pointer
- the current size
- the current capacity

The actual elements live somewhere else. Moving the vector simply moves those three values. No elements are moved. The pointer is simply transferred. That's why moving a `std::vector` is usually an **O(1)** operation.

`std::inplace_vector` can't do that.
There isn't a pointer to transfer. The elements are embedded directly inside the

object.

```
std::inplace_vector<int, 8> a = {1,2,3};
std::inplace_vector<int, 8> b = std::move(a);
```

Every stored element has to be moved individually. That makes move construction and move assignment **linear** in the number of elements.

If you're used to thinking of move operations as "**free,**" this is one of the biggest behavioral differences.

. . .

Performance Considerations

One of the biggest advantages of `std::inplace_vector` is predictable performance. Since the storage is part of the container itself, elements are never relocated because the container needs to grow. As long as there is remaining capacity, inserting a new element simply constructs it in the next available slot.

This also eliminates the cost of heap allocations and reallocations that `std::vector` may incur as it grows. For latency-sensitive or real-time applications, avoiding those unpredictable allocation costs can be just as important as raw performance.

Like `std::vector`, elements are stored contiguously, making iteration cache-friendly. Because `std::inplace_vector` embeds its storage directly within the container, it also benefits from excellent **memory locality**. Modern CPUs fetch memory in cache lines, so accessing one element often brings neighboring elements into the CPU cache as well, reducing cache misses during sequential traversal.

Of course, these benefits come with a trade-off. The storage for all `N` elements is reserved as part of the object itself, so choosing an unnecessarily large capacity increases the size of every `std::inplace_vector` instance.

. . .

What Does a Moved-From `inplace_vector` Look Like?

One surprisingly controversial topic in the proposal was the state of a moved-from container.

- Should it become empty?
- Should it keep the same size?
- Should something else happen?

The committee considered all three options.

> *A moved-from* **`std::inplace_vector`** *is left in a* ***valid but unspecified state****.*

In other words, you can safely destroy it. You can assign a new value to it. You can clear it. But you shouldn't make assumptions about what's still inside.

```
std::inplace_vector<std::string, 4> a;
```

```
a.push_back("Alice");
a.push_back("Bob");

auto b = std::move(a);

// a is valid...
// ...but don't assume anything about its contents.
```

That's almost identical to the guidance for many other standard library types.

There Is One Interesting Exception:
Things change if the element type is trivially copyable.
Consider this.

```
std::inplace_vector<int, 8>
```

Moving integers doesn't require calling move constructors. The compiler can simply copy bytes. Because of that, the committee allows implementations to use trivial move operations.

That means a moved-from container may still have the same size. This sounds unusual, but it enables something very valuable. The container itself can become trivially copyable.

. . .

Trivially Copyable Containers

One of the design goals of the proposal was supporting high-performance computing.

Imagine a container like this:

```
std::inplace_vector<float, 16>
```

If the container itself is trivially copyable, moving it between memory regions becomes extremely cheap.

That matters in areas like:

- GPU programming
- shared memory
- serialization
- high-performance networking

- MPI communication

Instead of copying sixteen individual objects, the implementation may simply copy the entire object as raw bytes.

That's something `std::vector` can never offer because the actual data lives somewhere else.

For certain workloads, this alone makes `std::inplace_vector` attractive.

. . .

Iterator Invalidation Is Different

Because `std::inplace_vector` never reallocates, you might assume iterators are always stable.
Not quite! Some operations still move elements around.

```
numbers.erase(numbers.begin());
```



```
auto it = numbers.begin();
std::swap(numbers, other);
```

- For `std::vector`, iterators remain valid after swapping.
  The underlying buffers are exchanged.
- For `std::inplace_vector`, that's impossible. The elements themselves are moved.
  Every iterator becomes invalid.

The same thing happens after moving the container.

```
auto it = values.begin();
auto other = std::move(values);
```

That iterator can no longer be used. It's one of the few places where `std::vector` and `std::inplace_vector` behave noticeably differently.

. . .

Why Isn't It Allocator-Aware?

This question naturally comes up.
Almost every standard container supports custom allocators.
Why not this one?
Because it doesn't actually allocate memory. The storage already exists inside the object. Adding allocator support would introduce complexity without solving an important problem.

The committee intentionally chose not to delay the proposal for a feature with limited practical value. That doesn't mean allocator support is impossible in the future. It simply wasn't considered worth the additional design complexity today.

Sometimes the simplest design really is the best one.

. . .

`constexpr` Gets More Interesting

One particularly nice property of `std::inplace_vector` is that it can be used during constant evaluation in situations where `std::vector` historically couldn't.

For suitable element types, this is perfectly valid.

```
constexpr auto create()
{
    std::inplace_vector<int, 4> values;
```

```
    values.push_back(1);
    values.push_back(2);
    values.push_back(3);

    return values;
}
```

That opens the door to compile-time algorithms that need a dynamically changing sequence but still have a known upper bound.

It's a niche feature, but one that simply wasn't practical before.

. . .

When Should You Actually Use It?

Just because `std::inplace_vector` exists doesn't mean it replaces `std::vector`.

Most of the time, it doesn't.

A good fit looks something like this:

- The maximum number of elements is known.
- Heap allocation is undesirable or unavailable.
- Predictable memory usage matters.
- You want contiguous storage.
- You still need a dynamically changing size.

Examples include:

- packet parsers
- embedded firmware
- game engines
- real-time systems
- compiler data structures

- GPU and accelerator code

Those are exactly the kinds of projects that have been shipping their own fixed-capacity vectors for years.

For everything else, `std::vector` is still the better default.

. . .

Final Thoughts

C++ already offers excellent containers for a variety of use cases. `std::array` is ideal when the number of elements is fixed for the lifetime of the object, while `std::vector` excels when the container needs to grow dynamically. `std::inplace_vector` sits comfortably between the two, combining a fixed maximum capacity with a runtime size that can grow and shrink without ever allocating memory dynamically.

It isn't intended to replace either `std::array` or `std::vector`. Instead, it fills a gap that C++ developers have been addressing with custom containers for years. If your application knows the maximum number of elements ahead of time but still needs the flexibility of a dynamically sized container, `std::inplace_vector` is likely the right tool for the job.

With C++26, that solution is finally part of the standard library, giving developers a well-designed, familiar, and portable alternative to the countless `static_vector` implementations that have existed across codebases for years.

. . .

Found this article helpful? Please **Clap** 👏 and **follow** for more C++ and system programming content.

Technology | Programming | Cpp26 | C Plus Plus Language | Software Development